

ui.clipboard 全面详解（基于 NiceGUI 官方文档）

`ui.clipboard` 是 NiceGUI 框架提供的剪贴板操作工具，支持**文本读写**、**图片读取**两种核心能力，同时提供服务器端 API 和客户端原生 API 两种调用方式，适配不同浏览器兼容性和性能需求。以下从核心功能、使用场景、代码示例、注意事项等方面展开详细说明：

一、核心功能概述

`ui.clipboard` 的核心作用是实现网页应用与系统剪贴板的交互，主要支持三类操作：

1. **文本写入**：将指定文本内容写入系统剪贴板；
2. **文本读取**：从系统剪贴板中读取文本内容；
3. **图片读取**：从系统剪贴板中读取图片内容（仅支持读取，不支持直接写入图片）。

其底层依赖浏览器的 Clipboard API，但 NiceGUI 对其进行了封装，提供了更简洁的 Python 接口，同时也支持直接调用客户端原生 API，兼顾灵活性和兼容性。

二、两种调用方式（含完整代码示例）

NiceGUI 提供了**服务器端封装 API** 和 **客户端原生 API** 两种使用方式，分别适用于不同场景：

方式 1：服务器端封装 API（简洁易用）

通过 `ui.clipboard.read()`、`ui.clipboard.write()`、`ui.clipboard.read_image()` 三个封装方法，直接在 Python 代码中操作剪贴板，无需编写 JS 代码，适合简单场景。

完整代码示例

```
from nicegui import ui

# 1. 写入文本到剪贴板
ui.button('Write Text', on_click=lambda: ui.clipboard.write('Hi!'))

# 2. 从剪贴板读取文本（异步操作，需用 async/await）
async def read_text() -> None:
    text = await ui.clipboard.read() # 异步读取文本
    ui.notify(f'Read text: {text}') # 弹出通知显示读取结果

ui.button('Read Text', on_click=read_text)

# 3. 从剪贴板读取图片（异步操作，返回图片数据URL）
async def read_image() -> None:
    img_data = await ui.clipboard.read_image() # 读取图片，返回数据URL
    if not img_data: # 若剪贴板中无图片，返回空值
        ui.notify('You must copy an image to clipboard first.')
    else:
        # 将读取到的图片显示在页面上（classes('w-72') 设置图片宽度为72rem）
        image.set_source(img_data)

# 初始化图片显示组件（初始无内容）
```

```
image = ui.image().classes('w-72')
ui.button('Read Image', on_click=read_image)

# 运行应用
ui.run()
```

关键说明

- **异步特性：**`read()` 和 `read_image()` 是异步方法，必须在 `async` 函数中通过 `await` 调用（`write()` 是同步方法，可直接调用）；
- **图片返回格式：**`read_image()` 成功读取后返回 **Data URL 格式** 的图片数据（如 `data:image/png;base64,...`），可直接作为 `ui.image()` 的数据源；
- **错误处理：**若剪贴板中无图片，`read_image()` 返回空值，需手动判断并提示用户。

方式 2：客户端原生 API（兼容性更强）

通过 `js_handler` 直接调用浏览器原生 `navigator.clipboard` API，避免与服务器的网络往返，且兼容性更广（部分浏览器对原生 API 支持更好，因剪贴板操作直接由用户点击触发）。

完整代码示例

```
from nicegui import ui

# 1. 客户端写入文本（直接调用 navigator.clipboard.writeText）
ui.button('Write Text (Client-side)', on='click', js_handler='''
    () => navigator.clipboard.writeText("Ho!") # 原生JS写入文本
''')

# 2. 客户端读取文本（通过 emitEvent 向服务器传递结果）
ui.button('Read Text (Client-side)', on='click', js_handler='''
    async () => {
        const text = await navigator.clipboard.readText(); // 原生JS读取文本
        emitEvent("clipboard_read", text); // 向服务器发送事件，携带读取结果
    }
''')

# 服务器监听客户端事件，接收并显示读取结果
ui.on('clipboard_read', lambda e: ui.notify(f'Client-side read: {e.args}'))

# 运行应用
ui.run()
```

关键说明

- **无网络往返：**所有剪贴板操作在客户端完成，无需与服务器通信，响应速度更快；
- **事件通信：**客户端读取结果需通过 `emitEvent` 发送给服务器，服务器通过 `ui.on()` 监听事件接收数据；
- **原生 API 限制：**客户端原生 API 同样需要用户交互触发（如点击按钮），否则浏览器会拒绝访问剪贴板（安全限制）。

三、浏览器兼容性与权限说明

1. 权限要求

- 剪贴板访问属于敏感操作，**必须由用户主动触发**（如点击按钮），否则浏览器会阻止操作（避免恶意网站窃取剪贴板内容）；
- 部分浏览器（如 Chrome、Firefox）会在首次访问时弹出权限请求，用户需允许后才能正常使用。

2. 功能支持限制

- 图片写入**：无论是封装 API 还是原生 API，均不支持直接将图片写入剪贴板（仅支持读取）；
- 旧浏览器兼容**：IE 等旧浏览器不支持 Clipboard API，需确保目标用户使用现代浏览器（Chrome 66+、Firefox 63+、Edge 79+ 等）；
- HTTPS 要求**：除本地开发环境（localhost）外，生产环境需通过 HTTPS 协议部署，否则浏览器会禁用剪贴板 API。

四、使用场景对比

调用方式	优势	劣势	适用场景
服务器端封装 API	无需编写 JS，Python 语法统一	需与服务器通信，有网络开销	简单文本 / 图片操作、快速开发场景
客户端原生 API	无网络往返，响应快、兼容性更好	需编写少量 JS，事件通信略繁琐	高性能需求、复杂交互场景、兼容性要求高

五、常见问题与解决方案

1. 读取剪贴板时无响应 / 报错

- 原因：未通过用户主动操作触发（如直接在脚本中调用，而非按钮点击事件）；
- 解决方案：确保所有剪贴板操作都绑定在用户交互事件上（如 `on_click`）。

2. `read_image()` 始终返回空值

- 原因：剪贴板中无图片数据，或复制的是图片文件路径而非图片本身；
- 解决方案：让用户直接复制图片（如在浏览器中右键复制图片、截图后复制），而非复制图片链接或文件。

3. 浏览器提示“权限被拒绝”

- 原因：用户拒绝了剪贴板访问权限，或网站未使用 HTTPS（生产环境）；
- 解决方案：引导用户允许权限，生产环境部署时启用 HTTPS。

六、总结

`ui.clipboard` 是 NiceGUI 中便捷的剪贴板操作工具，提供了两种灵活的调用方式，覆盖文本读写、图片读取核心需求。使用时需注意：

1. 所有操作必须由用户主动触发（如点击按钮）；
2. 兼容现代浏览器，生产环境需 HTTPS 部署；
3. 根据性能和开发效率需求选择调用方式（简单场景用封装 API，高性能场景用原生 API）。

通过合理选择调用方式并处理边界情况（如权限拒绝、无数据），可实现稳定、流畅的剪贴板交互体验。